

diveXplore - An Open-Source Software for Modern Video Retrieval with Image/Text Embeddings

Mario Leopold
Institute of Information Technology
Klagenfurt University
Klagenfurt, Austria
mario.leopold@aau.at

Farzad Tashtarian
Institute of Information Technology
Klagenfurt University
Klagenfurt, Austria
farzad.tashtarian@aau.at

Klaus Schoeffmann
Institute of Information Technology
Klagenfurt University
Klagenfurt, Austria
klaus.schoeffmann@aau.at

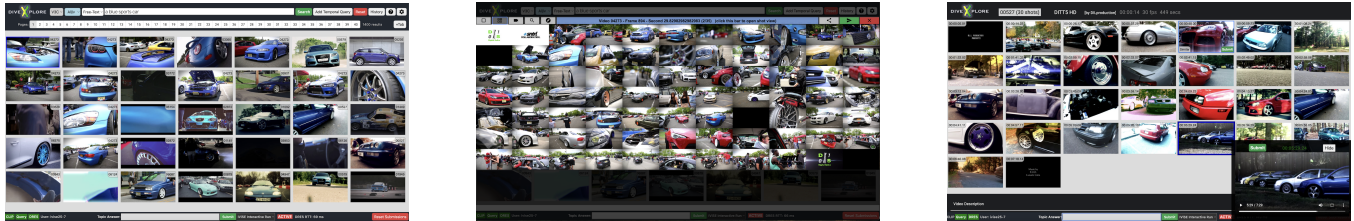


Figure 1: (a) Keyframes shown after retrieving results for the free-text query: “a blue sports car”. (b) Enhanced keyframe preview after clicking on a keyframe from the query results. (c) Shotlist of the video 00527 from the V3C dataset.

Abstract

Effective video retrieval in large-scale datasets presents a significant challenge, with existing tools often being too complex, lacking sufficient retrieval capabilities, or being too slow for rapid search tasks. This paper introduces diveXplore, an open-source software designed for interactive video retrieval. Due to its success in various competitions like the *Video Browser Showdown (VBS)* and the *Interactive Video Retrieval 4 Beginners (IVR4B)*, as well as its continued development since 2017 [11], diveXplore is a solid foundation for various kinds of retrieval tasks. The system is built on a three-layer architecture, comprising a backend for offline preprocessing, a middleware with a Node.js and Python server for query handling, and a MongoDB for metadata storage, as well as an Angular-based frontend for user interaction. Key functionalities include free-text search using natural language, temporal queries, similarity search, and other specialized search strategies. By open-sourcing diveXplore, we aim to establish a solid baseline for future research and development in the video retrieval community, encouraging contributions and adaptations for a wide range of use cases, even beyond competitive settings.

CCS Concepts

• **Information systems** → Users and interactive retrieval; Search interfaces; • **Human-centered computing** → Visualization systems and tools.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

MM '25, Dublin, Ireland

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2035-2/2025/10

<https://doi.org/10.1145/3746027.3756877>

Keywords

video retrieval, video analysis, video interaction

ACM Reference Format:

Mario Leopold, Farzad Tashtarian, and Klaus Schoeffmann. 2025. diveXplore - An Open-Source Software for Modern Video Retrieval with Image/Text Embeddings. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, October 27–31, 2025, Dublin, Ireland. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3746027.3756877>

1 Introduction and Motivation

Effective video retrieval in large-scale video datasets is challenging, as systems often are either too complex, offer not enough retrieval capabilities, or are too slow for fast retrieval tasks. Benchmarking competitions, such as the *Video Browser Showdown (VBS)* [10] and the *Interactive Video Retrieval 4 Beginners (IVR4B)* [1], aim to fuel research in this field, as they test the tools’ efficiency, speed, and usability in a competitive setting.

Our software, diveXplore, was initially developed in 2017 to participate in such video retrieval competitions [11]. Since then, it has undergone several optimizations and reimplementations, refining its feature set and optimizing its usability. At VBS 2024, diveXplore received an award for *Best Video Question-Answering-Tool for Novices*¹. The tool aims to provide a broad range of retrieval capabilities, as well as browsing functionalities, allowing users to explore and retrieve video segments from various sources. Figure 1 shows the diveXplore interface, including the main querying view, the shot-view, and the preview window that has several sub-views (e.g., a video summary and a video player). As diveXplore is designed to work with both generic (like the V3C dataset [6]) and highly specialized datasets (like MVK [14]), it covers a broad range of retrieval possibilities.

While diveXplore proves to be a competitive tool for video retrieval competitions, we share the work on diveXplore with the

¹<https://videobrowsershowdown.org/hall-of-fame>

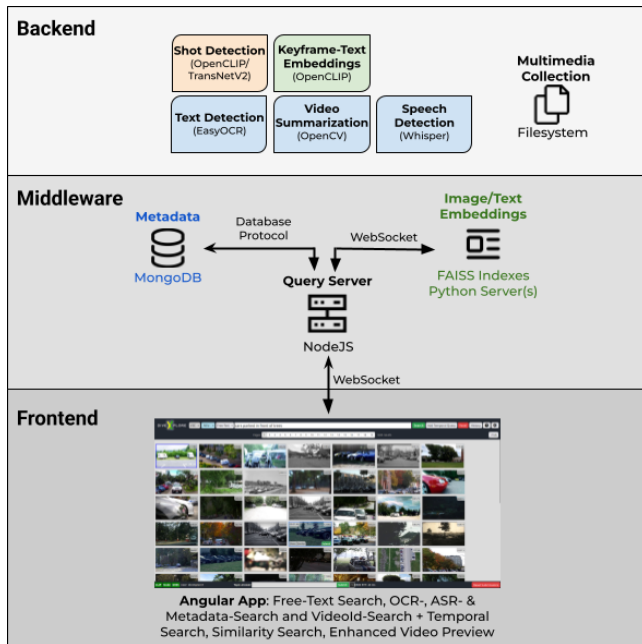


Figure 2: Three-layer architecture of diveXplore, consisting of a backend, middleware, and frontend.

scientific community to allow for others to contribute to the tool or adapt it for other use cases. By publishing diveXplore under an open-source license, future competitors can use it as a baseline for their implementations or adapt our contributions into their own video retrieval software.

2 System Architecture

diveXplore is comprised of a three-layer architecture, split into a backend, middleware, and frontend. It is freely available on GitHub². The complete architecture with all important components can be seen in Figure 2.

2.1 Backend

diveXplore’s backend handles all of the offline preprocessing needed to analyze and store the information extracted from the video datasets. Figure 2 shows the different tools used during preprocessing. The preprocessing steps include:

Shot Detection & Keyframe Extraction. Shot detection is done for each video via `shotdetection.py` by sampling a frame from every second of the video and extracting its features using OpenCLIP [2] (model ViT-H-14, trained on Laion2B [12]). The one-second interval was chosen to achieve a balance between granularity and processing efficiency. Future work could explore adaptive interval selection based on scene dynamics or confidence thresholds in feature change to further optimize the shot detection algorithm. Afterwards, we calculate the Manhattan distance of the features of successive sampled frames. If this distance exceeds a predefined threshold, the keyframe is marked as the beginning of a new shot.

²<https://github.com/klschoef/divexplore>

In the end, we extract a keyframe from the middle of each shot and store it together with a `.scenes.txt` file containing the respective shot information.

Alternatively, one could also employ TransNetv2[13] for shot detection, which is not integrated in this software though, and needs to be installed externally. It should be noted though that diveXplore is compatible with the output of TransNETv2, since it uses the same file structure in the `.scenes.txt` file.

Text Detection. To extract text from the stored keyframes, we use EasyOCR³ by JaidedAI. As it supports over 80 languages, it is a great fit for our use case, as the datasets used during the competition contain a variety of different languages (`extract_ocr.py`).

Speech Detection. Using Whisper [5] by OpenAI, we extract text from spoken segments and map them to the nearest related keyframe (`extract_ASR.py`). Similar to the text detection library, Whisper offers good multilingual capabilities, officially supporting 57 different languages.

Keyframe-Text Embeddings. Using OpenCLIP ViT-H-14, trained on Laion2B, we extract embeddings from each keyframe and store them in an index file (`extract_openclip_image_features.py`). The generated index file is then used by the FAISS [3] index server, as described in Section 2.2, to perform similarity search.

After executing all of the preprocessing steps, a Python script (`createshotinfo.py`) is used to combine the extracted information and store it in a preconfigured MongoDB. Additionally, post-processing scripts to generate the thumbnails and video summaries are executed (`create_thumbnails.py`, `create_summary.py`).

2.2 Middleware

diveXplore’s middleware consists of a Node.js server for query translation, redirecting, and submission handling. An additional Python server is hosting the FAISS indices with the image embeddings, generated during the preprocessing steps in the backend. This server performs similarity search between the extracted features of a query (image or free-text) by calculating the cosine similarity between the stored FAISS entries and the passed query. An additional MongoDB in the middleware acts as storage for keyframe references, video information (e.g. frames per second, title, description), as well as the extracted data from the *Optical Character Recognition (OCR)* and *Automatic Speech Recognition (ASR)*.

When a user submits a query via the frontend, the Node.js server either forwards it to the FAISS index server via a WebSocket connection or translates it into a MongoDB query and pulls the data from the prehosted database, depending on the query type. The results are then returned to the frontend as a list of relevant keyframes, together with essential information like timestamps, video title, and the number of retrieved results.

As multiple users might use the same system during competitions, we added limited collaboration functionalities to allow users to share keyframes and submissions. When a user connects to the Node.js server for the first time, a unique ID is assigned to them. A list of all the currently connected users is stored on the server. When one of the shared functionalities is used, it is broadcast to all others sharing the same queryserver. The available options for this functionality are keyframe sharing, where a user can press the

³<https://github.com/JaidedAI/EasyOCR>

share button in a keyframe’s video preview. This opens a popup on all connected users’ screens, showing the shared keyframe together with its video ID and the query used to retrieve it. Additionally, an opt-in feature enabled in the settings menu enables an overlay over all keyframes already shared by other users to avoid duplicate submissions, if desired.

2.3 Frontend

diveXplore’s frontend is built with Angular, offering various video retrieval and browsing capabilities. Additionally, we implemented an interface for seamless interaction with a DRES submission server. As diveXplore was originally designed to be used at the VBS and the IVR4B, different input fields and submission functionalities are part of the interface throughout most of the sites. None of the competition-related logic interferes with the core functionalities of the retrieval and browsing system, allowing users to manually remove the competition logic, if desired. The default retrieval capabilities of the diveXplore software include:

Free-Text Search. Allows the user to query with textual descriptions or keywords using natural language. After submission, embeddings are generated by OpenCLIP, which are then used to perform a cosine similarity search on the FAISS index server in the middleware. Figure 1(a) shows the displayed retrieval results after passing the free-text query “a blue sports car”. Apart from performing normal free-text queries, users can add additional input fields that are temporally (alternatively, the less-than < symbol can be used in the single query field). Such temporal queries retrieve video segments conforming to the queries in the specified order.

OCR-, ASR- & Metadata-Search. Offers the user the possibility to search for text, speech, and stored metadata across all keyframes. The passed query is then used to perform a regex-based search in the MongoDB, retrieving all instances of keyframes with the respective OCR-text, speech, and metadata stored in the database entry.

VideoId-Search. Expects a stored video-filename and retrieves all keyframes originating from the specified video.

As diveXplore was built for use with the VBS datasets (V3C [6], MVK [15], and LHE), choosing another dataset preset changes the available query modes. When choosing the MVK preset, all query modes except ASR-search can be selected. For the LHE-dataset, the same restrictions apply, but an additional query mode is added to the selection. *Prediction* queries offer the possibility to query for a specific instrument or organ, which are also stored in the MongoDB and retrieved via regex-based queries. This querying functionality was added to allow for more efficient video retrieval in the highly specialized LHE dataset. Apart from the querying options, diveXplore offers several browsing capabilities to analyze a video segment’s content. After passing a query and retrieving a set of keyframes, the user can click on a keyframe, opening the *enhanced video preview*. This preview window opens as a pop-up over the original window. Figure 1(b) shows the preview after clicking on a keyframe. In this preview, the user can switch between several windows, with each of them providing different information on the video from which the keyframe originated:

Enlarged Keyframe & Video Player. The default tab after selecting a keyframe opens a higher-resolution version of the selected

keyframe. The video tab opens a simple video player, starting playback from the selected keyframe.

Video Summary & Explore View. The video summary tab shows a collage of keyframes representing different parts of the respective shot. By scrolling the mouse wheel up or down, the user can increase or decrease the number of keyframes seen in the summary. The Explore View shows the summaries of videos that are visually similar to the selected video.

Shotlist. The shotlist presents all of the keyframes stored for the chosen video in temporal order. By default, the number of keyframes shown at once is limited to reduce the burden on the user’s system and browser resources. In addition to this, however, diveXplore also provides an own shot-view (via a linked page), which shows all extracted keyframes with their corresponding timestamps, an additional video player to inspect each shot, and video metadata – Figure 1(c) shows the shotlist of video 00527 from the V3C dataset.

When hovering over a retrieved keyframe, additional options are revealed as an overlay over the keyframe. The user can then see the timestamp in hh:mm:ss format, the video id, and buttons labeled *shot*, *submit*, and *similar*. The *shot* button redirects the user to the shot-view. Also, the user can perform similarity queries, submit keyframes, or browse through the available keyframes. The *similar* button sends the chosen keyframe to the middleware to perform a similarity search. After the response by the server, the retrieval results refresh to show similar keyframes based on the results retrieved by the FAISS index server.

Besides the user interaction and query forwarding to the middleware, the frontend handles the connection to the distributed evaluation server (DRES) [7–9] used during the competitions that diveXplore competed in. Submissions are automatically formatted according to the specifications of DRES and sent to the server, awaiting a response (using the OpenAPI specification of DRES).

3 Analysis Pipeline

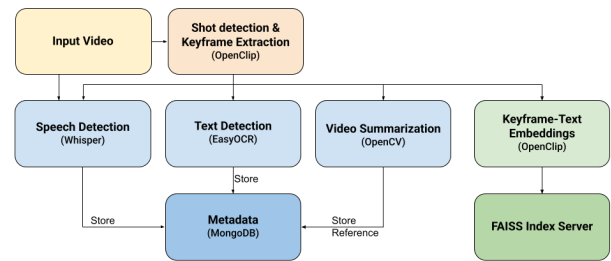


Figure 3: diveXplore’s preprocessing pipeline to extract the data needed by the middleware.

Analysis for video files is done in the backend of the three-layer architecture of diveXplore, as explained in Section 2.1. Figure 3 shows the steps needed to add a video or video dataset to the diveXplore software.

First, the video is passed to our shot detection and keyframe extraction algorithm, utilizing OpenCLIP. As already mentioned, the output of this step is a `.scenes.txt` file, containing the shot

boundaries (shot start and shot end) as well as a set of keyframes for each video, whereas each keyframe represents the middle of a shot. Shots are defined as separate scenes, split by either camera cuts or significant scene changes, determined by exceeding a predefined threshold when calculating the cosine similarity between successive keyframes' embeddings.

The resulting keyframes are then passed to several different scripts. The video summarization script generates different-sized grids of keyframes, whereas smaller grids show a coarser representation of the video, and larger grids a more fine-grained representation of it. The OCR script takes in the keyframes and extracts the detected text by EasyOCR from each of the keyframes. The ASR script expects the input video, as well as the shot information, to generate a transcript of the detected speech and map it to the relevant shots. All of the generated information is then stored in the middleware's MongoDB to be queried for during retrieval tasks.

Furthermore, embeddings for each image are extracted utilizing OpenCLIP. These embeddings are then used by the FAISS index server in the middleware to enable similarity searches based on calculating the cosine similarity between the stored indices and the generated embeddings from the query text or image.

Other than the mentioned analysis components, action detection and object detection algorithms are used to extract additional information from specialized datasets like the MVK or LHE dataset. As these are specific to the VBS, we excluded these analysis steps from Figure 3.

After finishing the analysis pipeline and launching the relevant middleware components, users can launch the frontend, connect it to the backend via a WebSocket connection, and start the retrieval tasks.

4 Use Cases

diveXplore was initially developed to compete at various video retrieval competitions, such as the VBS and the IVR4B. These competitions focus on testing the usability, efficiency, and effectiveness of video retrieval tools. As new competitors enter the competitions each year, utilizing state-of-the-art approaches for video retrieval is mandatory to stay competitive. Therefore, diveXplore received yearly iterations, learning from the previous years' results, and integrating new state-of-the-art features and models.

In addition to the competition-related version, a fork of diveXplore named presXplore was developed. This tool offers a different preprocessing pipeline, automating the steps needed to analyze and prepare the information extracted from given videos. Instead of focusing on datasets relevant to the VBS and IVR4B, presXplore is tailored to be used for presentation and lecture videos. To achieve this, we refined the shot detection algorithm to better accommodate slide changes happening during lectures and presentations. Additionally, all competition-related logic was stripped from the source code, and certain query types were removed.

The full source code of presXplore was published under the GPL-3.0 license on GitHub⁴. As shown by the evaluation of presXplore in [4], the diveXplore software can be adapted for other use cases, unrelated to benchmarking competitions.

⁴<https://github.com/marleo/PresXplore>

diveXplore could also be adapted for additional use cases, such as retrieval in private datasets or as an effective video browsing tool in various datasets. Due to the modularity of the preprocessing steps in the backend, modules like text detection or speech detection can easily be exchanged with different models. Additionally, future work could expand the module selection in the backend to accommodate other retrieval approaches or to extract additional data from the input videos.

5 Conclusion

We present diveXplore, an open-source video retrieval software used in various retrieval benchmark competitions since 2017. Receiving yearly iterations, this tool has been iteratively refined to improve usability, efficiency, and effectiveness. As diveXplore continually achieved high rankings in the competitions and even received rewards for its usability, it offers a great baseline for future developments in these tools.

Apart from using diveXplore as a system in competitive scenarios, it offers solid browsing capabilities, broadening the use cases it can be used in. Derivatives like the presXplore system present an example of how diveXplore can be used in specialized domains without the need for significant rework. Due to the three-layer architecture and the offline preprocessing, models like OpenCLIP or EasyOCR can be easily replaced if needed.

By publishing the system under an open-source license, we hope that others continue to develop the software and use it either for their own purpose, or for participating in the Video Browser Showdown competition.

References

- [1] Association for Computing Machinery 2023. *CBMI '23: Proceedings of the 20th International Conference on Content-based Multimedia Indexing* (Orleans, France). Association for Computing Machinery, Association for Computing Machinery, New York, NY, USA.
- [2] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. 2022. Reproducible scaling laws for contrastive language-image learning. doi:10.48550/ARXIV.2212.07143
- [3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]
- [4] Mario Leopold and Klaus Schoeffmann. 2025. An Open-Source Presentation Video Retrieval System Using Transformers. In *International Conference on Frontiers of Artificial Intelligence, Ethics, and Multidisciplinary Applications*. Springer. To appear.
- [5] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*. PMLR, 28492–28518.
- [6] Luca Rossetto, Heiko Schuldt, George Awad, and Asad A Butt. 2019. V3C—a research video collection. In *MultiMedia Modeling: 25th International Conference, MMM 2019, Thessaloniki, Greece, January 8–11, 2019, Proceedings, Part I* 25. Springer, 349–360.
- [7] Loris Sauter, Ralph Gasser, Abraham Bernstein, Heiko Schuldt, and Luca Rossetto. 2022. An asynchronous scheme for the distributed evaluation of interactive multimedia retrieval. In *Proceedings of the 2nd International Workshop on Interactive Multimedia Retrieval*. 33–39.
- [8] Loris Sauter, Ralph Gasser, Heiko Schuldt, Abraham Bernstein, and Luca Rossetto. 2024. Performance Evaluation in Multimedia Retrieval. *ACM Trans. Multimedia Comput. Commun. Appl.* 21, 1, Article 23 (Dec. 2024), 23 pages.
- [9] Loris Sauter, Ralph Gasser, Heiko Schuldt, Abraham Bernstein, and Luca Rossetto. 2024. Performance Evaluation in Multimedia Retrieval. *ACM Transactions on Multimedia Computing, Communications and Applications* 21, 1 (2024), 1–23.
- [10] Klaus Schoeffmann. 2019. Video browser showdown 2012-2019: A review. In *2019 International conference on content-based multimedia indexing (CBMI)*. IEEE, 1–4.
- [11] Klaus Schoeffmann, Manfred Jürgen Primus, Bernd Muenzer, Stefan Petschmann, Christof Karisch, Qing Xu, and Wolfgang Huerst. 2017. Collaborative feature

- maps for interactive video search. In *MultiMedia Modeling: 23rd International Conference, MMM 2017, Reykjavik, Iceland, January 4–6, 2017, Proceedings, Part II* 23. Springer, 457–462.
- [12] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. 2022. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in Neural Information Processing Systems* 35 (2022), 25278–25294.
- [13] Tomáš Souček and Jakub Lokoc. 2024. Transnet v2: An effective deep network architecture for fast shot transition detection. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 11218–11221.
- [14] Quang-Trung Truong, Tuan-Anh Vu, Tan-Sang Ha, Jakub Lokoč, Yue-Him Wong, Ajay Joneja, and Sai-Kit Yeung. 2023. Marine video kit: a new marine video dataset for content-based analysis and retrieval. In *International Conference on Multimedia Modeling*. Springer, 539–550.
- [15] Quang-Trung Truong, Tuan-Anh Vu, Tan-Sang Ha, Jakub Lokoč, Yue Him Wong Tim, Ajay Joneja, and Sai-Kit Yeung. 2023. Marine Video Kit: A New Marine Video Dataset for Content-based Analysis and Retrieval. In *MultiMedia Modeling - 29th International Conference, MMM 2023, Bergen, Norway, January 9–12, 2023*. Springer.